Modül 25

# PrimeNG Derinlemesine

## TSE'nin UI Library'si

| | |
|---|---|
| **Modül:** | 25 / 30 |
| **Ders Sayısı:** | 8 ders |
| **Seviye:** | Orta-İleri |
| **Versiyon:** | Angular v21 |

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

# Modül 25'e Hoş Geldin

PrimeNG — TSE'de günlük kullandığın UI library. Bu modülde en sık kullanılan komponentleri derinlemesine işliyoruz, theming yapıyoruz, performance tips veriyoruz, ve kendi component'lerini nasıl yazacağını gösteriyoruz.

Öğrenecekler:

- ✓ PrimeNG v20+ kurulum ve setup
- ✓ DataTable derinlemesine (sort, filter, paginate, lazy)
- ✓ Dialog, Toast, ConfirmDialog patterns
- ✓ Form components (Calendar, Dropdown, AutoComplete)
- ✓ FileUpload component
- ✓ Theming ve TSE branding
- ✓ Custom styling ve dark mode
- ✓ Performance (virtual scroll, lazy load)

DERS 25.1

# PrimeNG Kurulum

Modern v20+ kurulum ve config.

## ⚙ Kurulum

```
npm install primeng primeicons @primeng/themes

# Veya tek seferde
npm install primeng@20 primeicons @primeng/themes
```

## ⚙ App Config

```
// app.config.ts
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { providePrimeNG } from 'primeng/config';
import Aura from '@primeng/themes/aura';

export const appConfig: ApplicationConfig = {
  providers: [
    provideAnimationsAsync(),
    providePrimeNG({
      theme: {
        preset: Aura,
        options: {
          prefix: 'p',
          darkModeSelector: '.dark-mode',
          cssLayer: {
            name: 'primeng',
            order: 'tailwind-base, primeng, tailwind-utilities'
          }
        }
      }
    }),
  ]
};
```

## Mevcut Theme'ler

| Theme | Karakter | Use Case |
|---|---|---|
| Aura | Modern, sade | Default, çoğu app |
| Material | Google MD | Android-like |

| | | |
|---|---|---|
| Lara | Klasik PrimeNG | Eski projeler |
| Nora | Minimal | Sade tasarım |

## Imports

```
// Standalone component'lerde
import { ButtonModule } from 'primeng/button';
import { TableModule } from 'primeng/table';
import { DialogModule } from 'primeng/dialog';

@Component({
  imports: [ButtonModule, TableModule, DialogModule],
  // ...
})
```

DERS 25.2

# p-table — DataTable Derinlemesine

Enterprise app'in en kritik komponenti.

## Temel Kullanım

```
import { TableModule } from 'primeng/table';

@Component({
  imports: [TableModule],
  template: `
    <p-table [value]="users()" [paginator]="true" [rows]="10">
      <ng-template pTemplate="header">
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Email</th>
          <th>Role</th>
        </tr>
      </ng-template>
      <ng-template pTemplate="body" let-user>
        <tr>
          <td>{{ user.id }}</td>
          <td>{{ user.name }}</td>
          <td>{{ user.email }}</td>
          <td>{{ user.role }}</td>
        </tr>
      </ng-template>
    </p-table>
  `
})
export class UsersTableComponent {
  users = signal<User[]>([]);
}
```

## Sort & Filter

```
<p-table
  [value]="users()"
  [globalFilterFields]="['name', 'email']"
  #dt>

  <ng-template pTemplate="caption">
    <input
      pInputText
      type="text"
      placeholder="Ara..."
      (input)="dt.filterGlobal($any($event.target).value, 'contains')">
  </ng-template>

  <ng-template pTemplate="header">
    <tr>
      <th pSortableColumn="name">
        Name <p-sortIcon field="name" />
      </th>
      <th pSortableColumn="email">
        Email <p-sortIcon field="email" />
      </th>
      <th>
        Role
        <p-columnFilter
          field="role"
          matchMode="equals"
          [showMenu]="false">
          <ng-template pTemplate="filter" let-value let-filter="filterCallback">
            <p-select
              [ngModel]="value"
              (onChange)="filter($event.value)"
              [options]="roleOptions"
              placeholder="Hepsi" />
          </ng-template>
        </p-columnFilter>
      </th>
    </tr>
  </ng-template>

  <!-- body... -->
</p-table>
```

## ⚡ Lazy Loading (Server-Side)

```
<p-table
  [value]="users()"
  [lazy]="true"
  [totalRecords]="totalRecords()"
  [paginator]="true"
  [rows]="20"
  (onLazyLoad)="loadData($event)">

  <!-- templates -->
</p-table>
```

```
@Component({...})
export class UsersTableComponent {
  users = signal<User[]>([]);
  totalRecords = signal(0);
  loading = signal(false);

  private api = inject(UserApiService);

  loadData(event: TableLazyLoadEvent) {
    this.loading.set(true);

    this.api.getUsers({
      page: (event.first ?? 0) / (event.rows ?? 20),
      size: event.rows,
      sortField: event.sortField as string,
      sortOrder: event.sortOrder === 1 ? 'asc' : 'desc',
      filters: event.filters,
    }).subscribe({
      next: res => {
        this.users.set(res.items);
        this.totalRecords.set(res.total);
        this.loading.set(false);
      }
    });
  }
}
```

DERS 25.3

# Dialog, Toast, ConfirmDialog

Bildirim ve etkileşim komponentleri.

## □ p-dialog — Modal

```
import { DialogModule } from 'primeng/dialog';

@Component({
  imports: [DialogModule],
  template: `
    <button (click)="visible.set(true)">Aç</button>

    <p-dialog
      header="Kullanıcı Detayı"
      [(visible)]="visible"
      [modal]="true"
      [closable]="true"
      [draggable]="false"
      [resizable]="false"
      [style]="{ width: '50vw' }"
      [breakpoints]="{ '960px': '75vw', '640px': '95vw' }">

      <div>Modal içerik buraya</div>

      <ng-template pTemplate="footer">
        <button pButton severity="secondary"
(click)="visible.set(false)">İptal</button>
        <button pButton (click)="save()">Kaydet</button>
      </ng-template>
    </p-dialog>
  `
})
export class UserDialogComponent {
  visible = signal(false);

  save() {
    // ...
    this.visible.set(false);
  }
}
```

## □ Toast — Bildirim

```
// app.html
<p-toast position="top-right" />

// app.config.ts
providers: [MessageService]

// Component
import { MessageService } from 'primeng/api';

@Component({...})
export class MyComponent {
  private toast = inject(MessageService);

  saveSuccess() {
    this.toast.add({
      severity: 'success',
      summary: 'Başarılı',
      detail: 'Kaydedildi',
      life: 3000
    });
  }

  showError() {
    this.toast.add({
      severity: 'error',
      summary: 'Hata',
      detail: 'Bir sorun oluştu',
      sticky: true
    });
  }
}
```

## ConfirmDialog — Onay

```
// app.html
<p-confirmDialog />

// providers
[ConfirmationService]

// Component
import { ConfirmationService } from 'primeng/api';

@Component({...})
export class MyComponent {
  private confirm = inject(ConfirmationService);

  deleteUser(id: string) {
    this.confirm.confirm({
      message: 'Bu kullanıcıyı silmek istediğine emin misin?',
      header: 'Onay',
      icon: 'pi pi-exclamation-triangle',
      acceptLabel: 'Evet, Sil',
      rejectLabel: 'Vazgeç',
      accept: () => {
        // Sil
      }
    });
  }
}
```

DERS 25.4

# Form Components

Calendar, Select, AutoComplete, InputText.

## 📅 Calendar (DatePicker)

```
import { CalendarModule } from 'primeng/calendar';

<p-calendar
  [(ngModel)]="selectedDate"
  dateFormat="dd/mm/yy"
  [showIcon]="true"
  [showButtonBar]="true"
  placeholder="Tarih seç"
  [minDate]="minDate"
  [maxDate]="maxDate"
  appendTo="body" />

<!-- Range selection -->
<p-calendar
  [(ngModel)]="dateRange"
  selectionMode="range"
  [readonlyInput]="true" />

<!-- Time picker -->
<p-calendar
  [(ngModel)]="dateTime"
  [showTime]="true"
  hourFormat="24" />
```

## 📋 Select (Dropdown)

```
import { SelectModule } from 'primeng/select';

<p-select
  [(ngModel)]="selectedRole"
  [options]="roles"
  optionLabel="label"
  optionValue="value"
  placeholder="Rol seç"
  [filter]="true"
  filterBy="label"
  [showClear]="true" />

<!-- TypeScript -->
roles = [
  { label: 'Admin', value: 'admin' },
  { label: 'User', value: 'user' },
  { label: 'Manager', value: 'manager' }
];
```

## AutoComplete

```
import { AutoCompleteModule } from 'primeng/autocomplete';

<p-autocomplete
  [(ngModel)]="selectedUser"
  [suggestions]="suggestions()"
  (completeMethod)="search($event)"
  [field]="'name'"
  [forceSelection]="true"
  placeholder="Kullanıcı ara"
  [minLength]="2"
  [delay]="300" />

// Component
search(event: AutoCompleteCompleteEvent) {
  this.api.searchUsers(event.query).subscribe(users => {
    this.suggestions.set(users);
  });
}
```

DERS 25.5

# FileUpload Component

Dosya yükleme işlemleri.

## Basic Upload

```
import { FileUploadModule } from 'primeng/fileupload';

<p-fileUpload
  name="file"
  url="/api/upload"
  accept="image/*,application/pdf"
  [maxFileSize]="10000000"
  [multiple]="true"
  [auto]="false"
  (onUpload)="onUploadSuccess($event)"
  (onError)="onUploadError($event)"
  chooseLabel="Seç"
  uploadLabel="Yükle"
  cancelLabel="İptal" />

// Component
onUploadSuccess(event: FileUploadEvent) {
  console.log('Uploaded:', event.files);
}
```

## Custom Upload

```
<!-- Advanced: tam custom upload -->
<p-fileUpload
  [customUpload]="true"
  (uploadHandler)="customUpload($event)"
  [multiple]="true">

  <ng-template pTemplate="content">
    <p>Dosyaları buraya sürükle</p>
  </ng-template>
</p-fileUpload>

// Component
customUpload(event: FileUploadHandlerEvent) {
  const formData = new FormData();
  event.files.forEach(file => formData.append('files', file));

  this.http.post('/api/upload', formData, {
    reportProgress: true,
    observe: 'events'
  }).subscribe(response => {
    // Progress + result handling
  });
}
```

DERS 25.6

# Theming — TSE Branding

Marka renklerini PrimeNG'e uygula.

## Theme Customization

```
import { definePreset } from '@primeng/themes';
import Aura from '@primeng/themes/aura';

const TsePreset = definePreset(Aura, {
  semantic: {
    primary: {
      50: '{indigo.50}',
      100: '{indigo.100}',
      200: '{indigo.200}',
      300: '{indigo.300}',
      400: '{indigo.400}',
      500: '{indigo.500}',   // Ana renk
      600: '{indigo.600}',
      700: '{indigo.700}',
      800: '{indigo.800}',
      900: '{indigo.900}',
      950: '{indigo.950}'
    },
    colorScheme: {
      light: {
        primary: {
          color: '{primary.600}',
          inverseColor: '#ffffff',
          hoverColor: '{primary.700}',
          activeColor: '{primary.800}'
        }
      },
      dark: {
        primary: {
          color: '{primary.400}',
          inverseColor: '{primary.950}',
          hoverColor: '{primary.300}',
          activeColor: '{primary.200}'
        }
      }
    }
  }
});

// app.config.ts
providePrimeNG({
  theme: { preset: TsePreset }
})
```

## 🌙 Dark Mode

```
@Component({
  template: `
    <button (click)="toggleDark()">
      {{ isDark() ? '☀ Light' : '□ Dark' }}
    </button>
  `
})
export class ThemeToggleComponent {
  isDark = signal(false);

  constructor() {
    // Sistem tercihini al
    const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
    this.isDark.set(prefersDark);

    effect(() => {
      document.documentElement.classList.toggle('dark-mode', this.isDark());
    });
  }

  toggleDark() {
    this.isDark.update(v => !v);
  }
}
```

DERS 25.7

# Custom Styling

PrimeNG'in CSS değişkenlerini override etmek.

## CSS Variables

```
/* styles.scss */
:root {
  /* Primary brand color */
  --p-primary-color: #1e3a8a;     /* TSE mavisi */
  --p-primary-contrast-color: #ffffff;
  --p-primary-hover-color: #1e40af;
  --p-primary-active-color: #1d4ed8;

  /* Surfaces */
  --p-surface-0: #ffffff;
  --p-surface-50: #f9fafb;

  /* Text */
  --p-text-color: #1f2937;
  --p-text-muted-color: #6b7280;

  /* Border */
  --p-content-border-color: #e5e7eb;

  /* Border radius */
  --p-border-radius-sm: 4px;
  --p-border-radius-md: 6px;
  --p-border-radius-lg: 8px;
}

.dark-mode {
  --p-surface-0: #1f2937;
  --p-text-color: #f9fafb;
  /* ... */
}
```

## Component-Level Styling

```
/* Sadece p-button override */
.p-button {
  font-weight: 600;
  letter-spacing: 0.025em;
}

.p-button.tse-primary {
  background: linear-gradient(135deg, #1e3a8a, #1e40af);

  &:hover {
    background: linear-gradient(135deg, #1e40af, #1d4ed8);
  }
}

/* Table'a TSE styling */
.p-datatable.tse-table {
  .p-datatable-header {
    background: var(--tse-header-bg);
    border-radius: 8px 8px 0 0;
  }

  .p-datatable-thead > tr > th {
    background: #f8fafc;
    color: #1e293b;
    font-weight: 600;
  }
}
```

## □ Tailwind ile Birlikte Kullanma

```
/* tailwind.config.js */
module.exports = {
  content: ['./src/**/*.{html,ts}'],
  plugins: [require('tailwindcss-primeui')],  // PrimeNG entegrasyonu
};

// Template'te birlikte
<p-button class="!px-8 !py-3 hover:!shadow-xl transition-all">
  TSE Button
</p-button>

<!-- ! ile Tailwind important = PrimeNG default'larını override -->
```

DERS 25.8

# Performance Tips

Büyük dataset'lerde PrimeNG performansı.

## ⚡ Virtual Scroll — Büyük Listeler

```
<p-table
  [value]="items()"
  [scrollable]="true"
  scrollHeight="500px"
  [virtualScroll]="true"
  [virtualScrollItemSize]="46">

  <!-- 10,000+ item'ı sorunsuz render eder -->
</p-table>
```

## Lazy Loading Best Practices

- ✓ Server-side pagination — 100+ row için zorunlu
- ✓ OnPush change detection — performans iyileşmesi
- ✓ trackBy kullan — DOM diff hızlanır
- ✓ columnHidden — gereksiz column'ları gösterme
- ✓ Async loading state — loading spinner ile UX

## Bundle Optimization

```
// ❌ Tüm PrimeNG'i import etme
import * as primeng from 'primeng';

// ✓ Sadece kullandığını import et
import { ButtonModule } from 'primeng/button';
import { TableModule } from 'primeng/table';
import { DialogModule } from 'primeng/dialog';

// Tree-shakeable — sadece kullanılanlar bundle'a girer
```

## Style Sharing (Micro-Frontend)

**TSE Pattern**

Tüm remote'ların aynı PrimeNG versiyonunu kullanması çok kritik. Aksi takdirde style conflict'leri olur. `federation.config.js` 'de PrimeNG'i singleton + strictVersion yap.

# Modül 25 Özeti

PrimeNG artık aklın gibi. TSE'de Flexi.Kalite, APM ve diğer uygulamalarında profesyonel kullanım için tüm araçlar elinde.

## Neler Öğrendin?

- ✓ PrimeNG v20+ modern setup
- ✓ DataTable (sort, filter, paginate, lazy)
- ✓ Dialog, Toast, ConfirmDialog patterns
- ✓ Form components (Calendar, Select, AutoComplete)
- ✓ FileUpload component
- ✓ Theming ile TSE branding
- ✓ Custom styling + Tailwind integration
- ✓ Performance (virtual scroll, lazy load)

**Sıradaki Modül**

Modül 26 — Real-Time Applications. WebSocket, SSE (Server-Sent Events), real-time data flows.